

3. Simple Factory Design Pattern in Java

3.1 Introduction

The **Simple Factory Pattern** is a **creational design pattern** that centralizes object creation logic in one place. It encapsulates the instantiation of classes and provides a single point of control to create objects without exposing the instantiation logic to the client.

It is not a part of the official GoF patterns but is widely used in practice.

3.2 Problem Without Factory

```
public class Client {  
    public static void main(String[] args) {  
        Shape shape = new Circle(); // hardcoded  
        shape.draw();  
    }  
}
```

Problems:

- Direct dependency on implementation
- Changing shape type means code change and recompilation

3.3 What is Simple Factory?

A simple factory:

- Has a static method to create and return objects
- Takes input to decide which object to create

```
public class ShapeFactory {  
    public static Shape createShape(String type) {  
        return switch (type.toLowerCase()) {  
            case "circle" -> new Circle();  
            case "rectangle" -> new Rectangle();  
            default -> throw new IllegalArgumentException("Unknown shape: " + type);  
        };  
    }  
}
```

3.4 Basic Implementation

Example1: Shape Interface and Implementations

```
package com.mannemlokes;

public interface Shape {  
    void draw();  
}
```

```
package com.mannemlokes;

public class Circle implements Shape {
    public void draw() {
        System.out.println("Drawing a Circle");
    }
}
```

```
package com.mannemlokes;

public class Rectangle implements Shape {
    public void draw() {
        System.out.println("Drawing a Rectangle");
    }
}
```

Factory Class

```
package com.mannemlokes;

public class ShapeFactory {
    public static Shape createShape(String type) {
        return switch (type.toLowerCase()) {
            case "circle" -> new Circle();
            case "rectangle" -> new Rectangle();
            default -> throw new IllegalArgumentException("Unknown shape: " + type);
        };
    }
}
```

Main Class

```
package com.mannemlokes;

public class Main {
    public static void main(String[] args) {
        Shape shape1 = ShapeFactory.createShape("circle");
        shape1.draw();

        Shape shape2 = ShapeFactory.createShape("rectangle");
        shape2.draw();
    }
}
```

Output:

```
Drawing a Circle
Drawing a Rectangle
```

3.5 Step-by-Step Examples

Beginner: Shape Factory (Above Example)

- Demonstrates polymorphism and decoupling

Example2: Intermediate: Notification Factory with Parameters

```
package com.mannemlokes;

public interface Notification {
    void send(String message);
}
```

```
package com.mannemlokes;

public class EmailNotification implements Notification {
    public void send(String message) {
        System.out.println("Email: " + message);
    }
}
```

```
package com.mannemlokes;

public class SMSNotification implements Notification {
    public void send(String message) {
        System.out.println("SMS: " + message);
    }
}
```

```
package com.mannemlokes;

public class NotificationFactory {
    public static Notification createNotification(String type) {
        return switch (type.toLowerCase()) {
            case "email" -> new EmailNotification();
            case "sms" -> new SMSNotification();
            default -> throw new IllegalArgumentException("Unknown notification
                                                                type: " + type);
        };
    }
}
```

```
package com.mannemlokes;

public class NotificationApp {
    public static void main(String[] args) {
        Notification email = NotificationFactory.createNotification("email");
        email.send("Welcome to the service!");

        Notification sms = NotificationFactory.createNotification("sms");
        sms.send("Your OTP is 123456");
    }
}
```

Output:

```
Email: Welcome to the service!
SMS: Your OTP is 123456
```

Example3: Advanced: Real-World Example - Document Reader

```
package com.mannemlokes;

public interface DocumentReader {
```

```
void read(String file);  
}
```

```
package com.mannemlokes;  
  
public class PDFReader implements DocumentReader {  
    public void read(String file) {  
        System.out.println("Reading PDF file: " + file);  
    }  
}
```

```
package com.mannemlokes;  
  
public class WordReader implements DocumentReader {  
    public void read(String file) {  
        System.out.println("Reading Word file: " + file);  
    }  
}
```

```
package com.mannemlokes;  
  
public class DocumentReaderFactory {  
    public static DocumentReader getReader(String fileType) {  
        return switch (fileType.toLowerCase()) {  
            case "pdf" -> new PDFReader();  
            case "word" -> new WordReader();  
            default -> throw new IllegalArgumentException("Unsupported file type:  
                                                         " + fileType);  
        };  
    }  
}
```

```
package com.mannemlokes;  
  
public class FileReaderApp {  
    public static void main(String[] args) {  
        DocumentReader pdf = DocumentReaderFactory.getReader("pdf");  
        pdf.read("report.pdf");  
  
        DocumentReader word = DocumentReaderFactory.getReader("word");  
        word.read("letter.docx");  
    }  
}
```

Output:

```
Reading PDF file: report.pdf  
Reading Word file: letter.docx
```

3.6 Pros and Cons

✓ Pros:

- Centralized object creation
- Encapsulates logic for instantiation
- Decouples client from concrete classes

✗ Cons:

- Still violates Open/Closed Principle (modifying factory to add new types)
- Becomes bloated if too many types are handled

3.7 When to Use / Not to Use

Use When:

- You need centralized object creation logic
- You expect to reuse creation logic across the app
- You want to hide instantiation details from the client

Avoid When:

- You want to adhere strictly to Open/Closed Principle
- You have too many types or subclasses to manage in one place

3.8 Summary

Aspect	Description
Goal	Encapsulate object creation
Interface	Common interface for product types
Factory	Central method to return appropriate object
Benefit	Decouples instantiation logic from clients

The Simple Factory Pattern is ideal for:

- Frameworks
- Libraries with plug-in behaviour
- Simplifying client-side object creation

While not a formal GoF pattern, it is a very practical and frequently used approach in real-world Java applications.